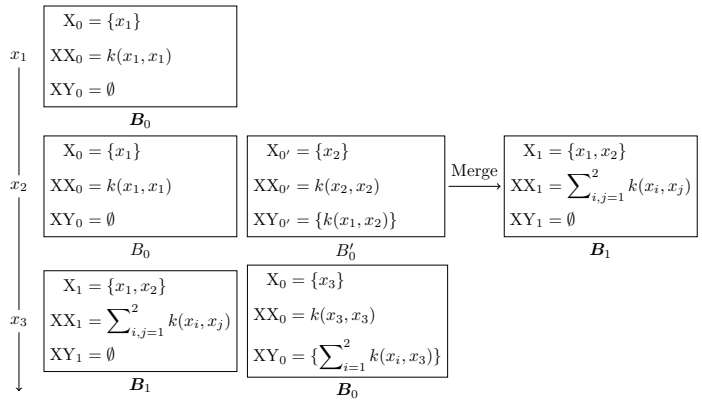

Figure 1: Schematic representation of Example 1. For a given step, the proposed scheme stores the windows in bold face.

Proof. For computing XX', we use the symmetry of k to obtain

$$\begin{aligned}\mathrm{XX}' &= \sum_{i,j=1}^{2^{s+1}} k\left(x_i^{(s+1)}, x_j^{(s+1)}\right) + \sum_{i,j=1}^{2^{s}} k\left(x_i^{(s)}, x_j^{(s)}\right) + \sum_{i=1}^{2^{s+1}}\sum_{j=1}^{2^{s}} k\left(x_i^{(s+1)}, x_j^{(s)}\right) + \sum_{i=1}^{2^{s}}\sum_{j=1}^{2^{s+1}} k\left(x_i^{(s)}, x_j^{(s+1)}\right) \\ &= \sum_{i,j=1}^{2^{s+1}} k\left(x_i^{(s+1)}, x_j^{(s+1)}\right) + \sum_{i,j=1}^{2^{s}} k\left(x_i^{(s)}, x_j^{(s)}\right) + 2\sum_{i=1}^{2^{s+1}}\sum_{j=1}^{2^{s}} k\left(x_i^{(s+1)}, x_j^{(s)}\right) = \mathrm{XX}_{s+1} + \mathrm{XX}_s + 2\mathrm{XY}_s^{s+1},\end{aligned} \tag{7}$$

which has a runtime complexity in $\mathcal{O}(1)$.

To compute XY', we note that B_{s+1} stores the list XY_{s+1} of kernel evaluations corresponding to all windows coming before it. The same holds for B_s, for which the list has one more element, XY_s^{s+1}, which was used in (7). All the elements in XY_s and XY_{s+1} are sums and thus additive; it suffices to merge both lists by adding their values element-wise, omitting XY_s^{s+1}, and storing the result in XY'. As each list has at most $\log t$ elements, merging them is in $\mathcal{O}(\log t)$. □

Specifically, the scheme facilitates the merging of windows of equal size, enabling us to establish the exponential structure outlined in the next section.

4.2.2 Insertion of observations

The structure is set up recursively. For each new observation, we create a new window B_0, with XX_0 as defined by (5) and XY_0 computed w.r.t. the already existing windows. If two windows have the same size, we merge them by Lemma 2, which costs $\mathcal{O}(\log t)$. This yields $\lfloor \log t \rfloor$ windows of exponentially increasing sizes.

We illustrate the scheme in the following Example 1 and the corresponding Figure 1.

Example 1. *To set up the structure, we start with the first observation x_1 and create the first window B_0, with XX_0 as defined by (5) and $\mathrm{XY}_0 = \emptyset$. When observing x_2, we similarly create a new window B'_0, now also computing $\mathrm{XY}_{0'}^0 = \{\mathrm{XY}_{0'}^0\}$. As B_0 and B'_0 have the same size, we merge them into B_1, computing XX_1 with (7). No previous window exists so that $\mathrm{XY}_1 = \emptyset$. We repeat this for all new observations, for example, for x_3, one creates (a new) B_0, computing XX_0 and $\mathrm{XY}_0 = \{\mathrm{XY}_0^1\}$, which results in two windows, B_1 and B_0.*

4.2.3 MMD computation and change detection

We now show that we can compute the MMD statistic (1) at positions between windows with a runtime complexity of $\mathcal{O}(\log t)$.

Proposition 2. *Let $B_l, \ldots, B_{s+1}, B_s, \ldots, B_0$ be a given list of windows with corresponding elements X_i, $i = 0, \ldots, l$, as defined in (4). For any split $s \in \{1, \ldots, l-1\}$, the computation of*

$$\mathrm{MMD}^2\left(\bigcup_{i=s+1}^{l} \mathrm{X}_i, \bigcup_{i=0}^{s} \mathrm{X}_i\right), \tag{8}$$

that is, the computation of MMD between the elements in windows coming before window B_s and the elements in windows coming after (and including) B_s, has a runtime complexity of $\mathcal{O}(\log t)$ for $0 < s < l$, with $s, l \in \mathbb{N}$.

Proof. To obtain (8), one recursively merges $B_s, \ldots, B_0$ to B'_s using Lemma 2, starting from the right, and similarly $B_l, \ldots, B_{s+1}$ to B'_l. One then obtains the statistic with Lemma 1, and by setting $\mathrm{XY}^{l'}_{s'} = \sum_{i=1}^{l-s} \mathrm{XY}^{i}_{s'}$, that is, by summing all elements in the XY'_s-list of B'_s. This concludes the proof as the logarithmic complexity was already established. □

The application of the presented data structure for change detection is as follows. For each new observation, we estimate MMD at any position between windows and compare it to the threshold $\epsilon'_\alpha = \frac{\epsilon_\alpha}{l}$ (with Bonferroni correction) from Proposition 1. We report a change when the value of MMD exceeds the threshold. As there are at most $\log t$ windows, we have at most $\log t - 1$ positions. Computing MMD for a position is in $\mathcal{O}(\log t)$ by Proposition 2, and so the procedure has a total runtime complexity of $\mathcal{O}\left(\log^2 t + t\right)$ per insert operation, where the term linear in t results from computing XY_0 when inserting a new observation. We may equivalently consider the proposed method as providing a more coarse-grained estimate of (3), taking the form

$$T' = \inf\left\{t : \max_{s=1,\ldots,l}\left[\mathrm{MMD}\left(\bigcup_{i=s+1}^{l} \mathrm{X}_i, \bigcup_{i=0}^{s} \mathrm{X}_i\right) \geq \epsilon_\alpha\right] = 1\right\}, \tag{9}$$

with the X_i-s defined as in Proposition 2.[5]

While the data structure in its current form allows to obtain the precise values of (1) in an incremental fashion, its runtime and memory complexity are $\mathcal{O}(t)$ for each new observation; these complexities are unsuitable for deploying the algorithm in the streaming setting. We reduce the runtime by subsampling within the windows, which we present together with the complete algorithm in the following section.

4.3 MMDEW Algorithm

Our algorithm builds upon the data structure discussed previously. But, we suggest that each window of size 2^s, $s = 0, \ldots, l$, samples s observations (of the total 2^s), that is, a logarithmic amount, while keeping everything else as before.

In this section, we first analyze such subsampling and discuss its benefits. Afterwards, we present the complete algorithm. We refer to Appendix A.2 for the proof of the following statement.

Proposition 3. *With subsampling, the number of terms in the sum XX_l for a window at position l, $1 \leq l$, $l \in \mathbb{N}$ is*

$$n_{\mathrm{XX}_l} = 2^{l-1}\left(l^2 - l + 4\right) = \frac{t}{2}\left(\log_2^2 t - \log_2 t + 4\right),$$

with $t = 2^l$ the number of observations of B_l. The number of terms of XY^l_l for windows of the same size, which occur prior to merging, is

$$n_{\mathrm{XY}^l_l} = 2^l l = t \log_2 t.$$

[5] Recall that Remark 1 gives the precise value of l. Further, some split positions s may not exist.

Input: Data stream $x_1, x_2, \ldots$, level α
Output: Change points in $x_1, x_2, \ldots$; detection times

1: *windows* $\leftarrow \emptyset$ ▷ List of windows
2: **for** each $x_i \in \{x_1, x_2, \ldots\}$ **do**
3: $X_0 \leftarrow x_i$ ▷ Initialize B_0
4: $\mathrm{XX}_0 \leftarrow k(x_i, x_i)$
5: **for** each $B_j \in$ *windows* **do**
6: $\mathrm{XY}_0^j \leftarrow \sum_{x_k^{(j)} \in B_j} k(x_i, x_k^{(j)})$
7: $B_0 = (\mathrm{X}_0, \mathrm{XX}_0, \mathrm{XY}_0)$
8: *windows* $\leftarrow$ *windows* $\cup B_0$
9: **for** each split s in *windows* $= \{B_l, \ldots, B_{s+1}, B_s, \ldots, B_0\}$) **do** ▷ Detect changes
10: **if** $\mathrm{MMD}\left(\bigcup_{j=s+1}^{l} \mathrm{X}_j, \bigcup_{j=0}^{s} \mathrm{X}_j\right) \geq \epsilon'_\alpha$ **then**
11: **print** "Change at s detected at time i"
12: *windows* $\leftarrow B_s, \ldots, B_0$ ▷ Drop windows
13: **while** two windows have the same size 2^l **do** ▷ Maintain exponential structure
14: Merge windows following Lemma 2 into B_{l+1}
15: Store a uniform sample of size $l+1$ in X_{l+1} of B_{l+1}

Algorithm 1: Proposed MMDEW change detection algorithm.

Remark 2. *The number of terms in the sums of* (1) *acts as a proxy for the quality of the estimate. It is optimal when no subsampling takes place; this number is* $\mathcal{O}\left(t^2\right)$*. When subsampling a logarithmic number of observations per window with our data structure (as we propose), one achieves polylogarithmic runtime and logarithmic memory complexity. At the same time, one achieves a better approximation quality than naively sampling a logarithmic number of observations without the summary data structure. While such sampling would also yield a memory complexity of* $\mathcal{O}(\log t)$ *when using the naive approach for change detection—that is, splitting the sample into two neighboring windows and computing* MMD^2*—the number of terms in* (1) *would be* $\mathcal{O}\left(\log^2 t\right)$*. Proposition 3 shows that the summary data structure improves upon this by a factor of approximately* $t/2$ *for* n_{XX_l} *and a factor of* $t/\log_2 t$ *for* $n_{\mathrm{XY}_l^l}$ *(we neglect logarithmic and constant terms in the former due to their small contribution).*

Algorithm 1 now summarizes the complete algorithm, with MMD in Line 10 referring to the computation of MMD as in Proposition 2. MMDEW stores only a uniform sample of size $l+1$, that is, of size logarithmic in the number of observations, while keeping the respective XX_s and XY_s, $s = 0, \ldots, l$, computed before. With this approach, the number of samples in a window increases by one each time the window is merged, and the memory complexity is logarithmic in the number of observations. Note that one recovers the previous algorithm (Section 4.2.3) and therefore the precise value of (8) if one omits Line 15. Further, changes in Line 15 allow to adjust the subsampling, for example, the user may defer the sampling until windows contain a minimum number of observations, or choose a different function to control the sample size.

The following example illustrates the procedure. Figure 2 expands upon Example 2 and shows the evolution of the data structure upon observing $x_1, \ldots, x_6$ and when merging windows.

Example 2. *We assume that there is a stream of i.i.d. observations* $x_1, x_2, \ldots$*. Note that the i.i.d. assumption implies that there are no changes. MMDEW receives the first observation,* x_1 *and creates a window* B_0 *storing* x_1*,* $\mathrm{XX}_0 = k(x_1, x_1)$*, and* $\mathrm{XY}_0 = \emptyset$*. For the next observation,* x_2*, it creates a new window* $B_{0'}$*, storing* x_2*,* $\mathrm{XX}_{0'} = k(x_2, x_2)$*, and* $\mathrm{XY}_{0'} = \{k(x_1, x_2)\}$ *and detects no change. As* B_0 *and* $B_{0'}$ *have the same size, MMDEW merges them into window* B_1*, storing a sample of size* $\log_2 2 = 1$*, say, it stores* x_1 *and discards* x_2*, and computes* $\mathrm{XX}_1 = k(x_1, x_1) + k(x_2, x_2) + 2k(x_1, x_2)$*, following* (5)*. As no previous window exists, the computation of* XY_1 *is not required. We see that the number of terms in* XX_1 *equals four, while* B_1 *stores only one observation (established in Proposition 3). Next, the algorithm observes* x_3 *and creates a new window,* B_0*, storing* x_3*,* $\mathrm{XX}_0 = k(x_3, x_3)$*, and computing* XY_0 *to the window coming before, that is,* B_1*, so that* $\mathrm{XY}_0 = \{\mathrm{XY}_0^1\}$*. In the next step, MMDEW receives* x_4*, again creating a new window* $B_{0'}$*.*